

fork () + COW + Resource Reference

不要问“复制了什么”，要逐对象追踪复制、共享、引用与后续分化

408 · OS-I02

Canonical Problem

父进程调用 `fork()` 后，为什么父子既“像复制了一份进程”，又会暂时共享物理页和打开文件状态？随后一方执行写内存、`read/lseek/close` 或 `exec()` 时，哪些关系继续共享，哪些在什么事件上分化？

Parent State $\xrightarrow{\text{fork}}$ New Child Identity + {per-object relations} $\xrightarrow{\text{later events}}$ Selective Divergence

本册拒绝用“fork 就是把整个进程复制一遍”作为终点。真正稳定的做法是：对每一种对象分别判断 Copy / Share / Reference / Rebuild，以及以后哪一个事件会改变这条关系。

Integration Ownership

本册 Owns `fork` 后跨 Process / VM / File 的组合轨迹与分化时点，不 Own 局部机制。

- OS-01/02 Own process identity、fork/exec 生命周期、父子返回值与调度；
- OS-B02 + OS-04 Own address-space association、private mapping、COW fault/retry；
- OS-B03 + OS-06/07 Own fd binding、OFD/open instance、file object 与 namespace lifetime；
- 若 `fork` 后继续发生 I/O，再按具体操作调用 OS-05；若进入共享内存并发正确性，再调用 OS-03。

本 Canonical Process 的核心不依赖 I/O 或同步机制，因此它们不被强行塞进主线。

1 第一原则：fork 产生的是新的进程身份，不是“所有对象都复制”

`fork()` 的核心语义是创建一个新的进程实例，使它从调用点附近继续执行，并继承父进程的一组环境关系。父子通过返回值区分当前控制流身份：

parent: child PID child: 0

因此第一层必须独立出来：

Process Identity_{parent} ≠ Process Identity_{child}

但“身份不同”并不推出所有底层资源立即拥有物理副本。`fork` 的设计恰好大量利用引用复制 + 延迟分化。

2 fork 后的关系向量：一次性把对象分层

对象/关系	fork 后稳定语义	常见高效实现	后续分化事件
Process identity	child 是新的进程实例，PID/父子关系独立	新建内核管理记录	exit/wait、独立调度生命周期
Execution context	父子都从 fork 返回附近继续，但返回值等状态不同	初始化 child 的可恢复执行现场	两边之后各自执行不同指令流

Address space identity	逻辑上是父子各自的地址空间环境	页表/region 关系复制或重建, 私有页可暂时共享底层内容	private write 触发 COW; exec 替换映像
Private writable page	写入结果应彼此隔离	初始共享物理内容 + 写保护/等价 COW 元数据	第一次需要独立写时 mapping 分化
Explicit shared mapping	共享语义继续存在	父子可继续引用同一共享 backing/page state	不是 COW 隔离; 由共享协议决定后续可见性
Descriptor table binding	child 得到父进程描述符绑定的语义副本	复制 fd binding 关系	close/dup/exec close-on-exec 改变各自绑定
Open instance / OFD	父子对应 fd 通常继续引用同一打开实例	增加引用关系, 不复制 current offset	read/write/lseek 可改变共享 open-state
File object	不因 fork 创建第二个文件对象	继续被相同 OFD 等引用	namespace/open references 的变化由 FS lifecycle 决定

这张表的核心不是记“复制/共享清单”，而是建立一个检查器：

Identity → Relation → Owner → Divergence Event

3 阶段一：fork 创建 child，并建立新的控制流身份

父进程进入内核执行 fork 创建流程。OS-01/02 负责：

1. 建立新的 child identity 和生命周期状态；
2. 初始化 child 的执行现场，使其未来被调度后能从 fork 返回路径继续；
3. 建立父子关系，并让父子在用户可观察返回值上不同；
4. 把 child 送入合适的 Ready/Runnable 状态，真正何时执行由 scheduler 决定。

因此：

fork success $\nRightarrow$ child immediately running

创建完成只是让它成为可被调度的新进程。

4 阶段二：地址空间是“逻辑独立 + 底层可暂时共享”

OS-B02 把最容易混淆的两层拆开：

- **Address-space semantics**：父子私有内存写入之后应彼此隔离；
- **Physical representation**：为了避免 fork 时复制所有页，多个 private mappings 可以暂时引用同一物理内容。

所以 COW 不是“fork 让父子共享内存”的最终语义，而是一种延迟兑现私有副本的 mechanism：

Private Semantics + Temporary Shared Representation $\xrightarrow{\text{write}}$ Selective Physical Divergence

若题目出现 MAP_SHARED/共享内存，则它本来就是 shared semantics，不能因为“fork 后两边都指到同一 frame”就把它误判成 COW private page。

5 阶段三：文件关系不是“文件复制”，而是 binding 复制后继续引用同一 OFD

对每个继承的 fd：

Parent fd binding → OFD Child fd binding → same OFD

这解释了一个经典结果：父子拥有各自的 fd 表绑定关系，但对应绑定继续引用同一个打开实例，因此该 OFD 拥有的 current offset 等状态是共享的。

例如文件当前 offset 为 0，父进程先通过共享 OFD 成功读取 100 bytes，使 current offset 变为 100；如果 child 随后使用其继承 fd 做普通顺序 read，它从同一 OFD 的当前位置继续。真正判据不是“fd 数字是否相同”，而是：

Do the bindings reference the same OFD?

若父子后来各自重新 open() 同一路径，通常会得到不同 OFD；它们可以引用同一 File Object，却拥有独立 current offset。这正是 B03 的三层模型。

6 四个后续事件：分化到底发生在哪一层

6.1 事件 A：父或子写 private page

写保护/fault 进入 OS-04：

private write → COW decision → 必要时准备私有页 → 更新写方 mapping → retry

只改变需要分化的 VM relation；PID、OFD、file object 不因此复制。

6.2 事件 B：一方 close(fd)

close 删除的是该进程的一条 descriptor binding，并相应减少对打开实例的引用。另一进程自己的 binding 仍在时，它继续使用同一 OFD。于是：

Child close ⇏ Parent fd invalid

6.3 事件 C：一方 read()/lseek()

若父子 binding 指向同一 OFD，改变 current offset 的操作会影响另一方后续基于该 offset 的操作；但这不等于它们共享全部进程状态，也不等于文件对象被复制/修改。

6.4 事件 D：一方 exec()

exec() 的稳定边界是：

same process identity, new program image

进程身份不因 exec 自动变成另一个进程；原程序映像/地址空间内容被新程序替换。打开描述符是否保留要看 close-on-exec 等接口状态。于是 fork+exec 常见路径正好展示两种不同 operation：fork creates identity; exec replaces image。

7 贯穿母例：父子各做一次内存写和文件读

假设父进程 P：

- private page 中变量 x=1;
- fd=3 指向 OFD-A, current offset=0;
- OFD-A 指向文件对象 F。

P 调用 fork 得到 child C 后:

1. P 与 C 的 process identity 已不同;
2. x 所在 private page 的逻辑地址空间已属两边, 但底层物理内容可以暂时共享;
3. P:fd3 与 C:fd3 是两条 descriptor binding, 二者继续引用 OFD-A;
4. C 执行 x=2: 只让 C 对该 private page 的 mapping 在 COW 时分化; P 仍看到自己的 private value;
5. P 通过 fd3 读 100 bytes: OFD-A 的 current offset 按实际读取量推进;
6. C 后续用继承 fd3 普通 read: 从 OFD-A 的新 current offset 继续; COW 的内存分化并不会让 OFD 自动分化。

这证明两个“共享/分化”轴彼此独立:

VM Mapping Relation ⊥ File Open-Reference Relation

一方发生 COW, 不会顺带复制 OFD; 一方 close fd, 也不会改变另一方 private page。

8 最容易错的五个绝对化说法

错误压缩	为什么错	稳定判据
“fork 把父进程完整复制一份”	把语义复制、引用复制和物理复制混在一起	逐对象标 Copy / Share / Reference / Rebuild
“父子共享地址空间”	私有映射语义要求后续写入隔离; 只是底层页可暂时共享	问 mapping semantics 与 physical representation
“父子 fd 表共享”	fd binding 属于各自进程; 共享的是绑定所引用的 OFD	画 Parent fd → OFD 与 Child fd → OFD
“相同 fd 数字就共享 offset”	两个独立 open 可用相同整数 fd 却指向不同 OFD	只看是否同一 OFD
“exec 创建新进程”	exec 替换程序映像, 不以创建新 PID 为定义	看 process identity 是否保持

9 扩展边界: 哪些细节不要偷进 408 主模型

- 多线程进程调用 POSIX fork() 时 child 继承哪些线程, 是接口级工程边界; 只有题目明确涉及才展开, 不反过来污染基础 process/VM/file 模型。
- COW 标志位放在哪里、页表层级怎样复制、是否原地复用唯一引用页、page size 多大, 属于 OS-04 具体实现。
- inode/dentry/OFD 的精确内核引用计数与 journaling 回收时序属于 OS-06/07。
- fork 后真正何时父先跑还是子先跑由调度决定; 没有题设不能根据代码顺序猜。

10 独立验证：做 fork 综合题时逐列验收

1. Identity：父子 PID/task identity 是否明确分开？

2. Execution：两条执行流分别从哪里继续，返回值如何区分？

3. VM：private/shared mapping 是否分开？物理共享是否只是实现表示？

4. File：fd binding、OFD、file object 是否三层分开？offset 属于哪一层？

5. Event：write/close/read/lseek/exec 分别改变哪条 relation，有没有错误“连带复制”？

6. Scheduling：child Ready 与 child Running 是否被分开？

Compression

以后看到 fork，不再问一句“复制了什么”，而是画一张关系表：

New Identity +

Execution Context: initialize/copy semantics

Private VM: logically separate, physically COW-share if allowed

Shared VM: keep shared semantics

fd binding: copy binding relation

OFD: referenced/shared

File Object: referenced, not duplicated

然后对后续每个事件只修改它真正拥有的那一层。这个模型比“整进程复制”更接近 408 题目和真实 OS。